Data generation: `engine/generate.py`

Builds the simulated cloud estate (inventory plus 30 days of 5-minute metrics) that the engine analyses. All constants come from `engine/config.py`. Prices are illustrative, as the UTHO brief allows.

1. What it does

Creates 96 VMs, 10 unattached volumes and 36 public IPs for an Indian SaaS company (regions Noida, Bangalore, Mumbai; teams Payments, Search, Platform, Data, Mobile-API, Growth). Each VM gets a workload persona, and its CPU, memory, disk, IOPS and network series are generated from that persona. The estate deliberately contains every pattern the brief lists: steady low-usage VMs, bursty VMs, batch jobs, dev environments idle on weekends, orphaned volumes and IPs, and a genuine anomaly.

Inputs: `config.py` (seed, window, plans, teams, regions). **Outputs:** `engine/data/metrics.npz` (float32 arrays, one per metric, shape VMs × 8,640) and `engine/data/inventory.json`.

2. Reproducibility

- One NumPy generator, `default_rng(42)`. Every run gives identical numbers.
- All random draws share this one generator, so the order of draws matters: adding or reordering a persona changes everything drawn after it.
- The memory leak (section 7) is written without random draws on purpose, so it does not shift any other series.

3. Time axis

- 30 days × 288 points (5-minute steps) = 8,640 points per metric.
- Window starts 2026-09-24 00:00 IST and ends 2026-10-23 23:55.
- For each point the script computes the fractional hour of day, the day number and a weekend flag.

4. Building blocks

Helper	Meaning
`noise(σ)`	Gaussian noise, mean 0, standard deviation σ
`bump(centre, width, height)`	Smooth Gaussian hump around an hour of the day (wraps past midnight)
`window(start, end, height)`	Flat block of the given height between two hours
`clip(x)`	Keeps percentages within 0–100

General shape of every series: `value(t) = base + daily shape + noise + events`.

5. Personas

Each persona returns CPU %, memory % and a "network scale" used to derive network and IOPS.

Persona	CPU %	Memory %	Net scale	Purpose
steady_low	6–10 base, small 14:00 bump (+3), noise σ1.5	22–30	0.6	Over-provisioned, low usage
healthy	base 38–52, 14:00 bump (+12), noise σ4	50–62	1.5	Right-sized control group
bursty	9 base, 30-minute spike (+70–76) at 10:00, 11:00 or 18:00	35, +20 during spike	1.0	The trap: low average, high peak
batch	2 base, 2-hour spike (+74–80) starting 01:00, 02:00 or 03:00	20, +45 during job	0.8	Idle most of the day, nightly job
office_dev	12–22 (+ bump 8 at 15:00) from 09:00 to 19:00 on weekdays, else 1.2	40 in work hours, 28 otherwise	0.5	Dev environment, idle nights and weekends
idle	1–1.8	12–18	0.02	Zombie / idle
memory_heavy	9, small bump	72–78 (+4 bump)	1.0	Memory-bound, must not be shrunk on RAM
under	72, 14:00 bump (+22), peaks near 94	70	2.0	Under-provisioned, should be upsized
growing_disk	same as healthy	same as healthy	1.5	Disk grows about 0.65% per day from 60%

Why these numbers: a 30-minute spike is 6 of 288 points (about 2.1% of the day), so it is visible to the p99 used by the rightsizing engine but invisible to an average. This is what the bursty persona exists to test.

6. Derived signals

With `activity = cpu / 100`:

- `iops = scale × 40 + activity × 900 × scale + noise(5)`, floored at 0
- `net_in = scale × (0.4 + activity × 2.5) + noise(0.03)`, floored at 0
- `net_out = scale × (0.3 + activity × 2.0) + noise(0.03)`, floored at 0
- `disk` = a constant between 30% and 65% per VM plus noise σ0.3; for growing_disk, `60 + day × 0.65 + noise(0.2)`

Every VM also gets: a random team and region, a name built from its environment and a team role (for example `prod-ledger-03`), a disk size from {50, 100, 200, 500} GB, and a launch date 40–700 days before the window (used later by `build.py` to rebuild the bill history).

7. The estate

Designed estate (85 VMs), set by the `DESIGN` table:

Persona	Count	Environments (count)
steady_low	18	prod 6, staging 5, dev 7
healthy	16	prod 10 + 6
bursty	5	prod
batch	6	prod 4, staging 2
office_dev	20	dev 15, staging 5
idle	8	dev 4, staging 2, prod 2
memory_heavy	6	prod 4, staging 2
under	4	prod
growing_disk	2	prod

Named showcase VMs (3):

- `prod-reports-01`: bursty, spike at 10:00. An average-based tool would shrink it to 2 vCPU; the engine refuses because its peak would exceed 100%.
- `dev-search-04`: steady_low, with CPU forced to about 1.4% at nights (20:00–08:00) and on weekends, so both resize and schedule recommendations apply.
- `prod-pay-api-07`: the genuine anomaly (section below).

Planted events:

Event	How it is generated	What it tests
Anomaly	From day 26.4, `prod-pay-api-07` CPU becomes 97 ± 1.5 and egress becomes 25 × its normal mean	Anomaly alert; sizing is paused for this VM
Cost jump	8 `s-16-64` ML VMs (`prod-ml-train-01..08`, Data team, Mumbai) launched on day 20; data is NaN before that	Daily cost jump alert (8 × ₹16,000 = ₹1.28 L/month at illustrative prices)
Memory leak	After day 12, memory on one prod `s-4-8` healthy VM grows 0.9% per day	CPU/memory trend alert
Metric gaps	VMs `vm-005`, `vm-031`, `vm-058` lose 1.5–4 days of data starting at a random day between 3 and 20	Confidence score reacts to missing data

Orphans:

- 10 unattached volumes: size from {100, 200, 250, 500} GB, environment drawn with probabilities dev 0.5, staging 0.3, prod 0.2, detached 9–120 days before the window.
- 36 public IPs: 30 attached to the first 30 prod VMs, and 6 reserved but unassigned (detached 10–90 days before the window).

Total: 85 + 3 + 8 = 96 VMs.

8. Output files

- `metrics.npz`: arrays `cpu`, `mem`, `disk`, `iops`, `net_in`, `net_out`, each VMs × 8,640, plus the VM id list.
- `inventory.json`: one record per resource (id, name, type, region, env, team, created_at, plan, vcpu, ram_gb, disk_gb, attached_to, detached_since, persona, notes).
- The `persona` label is removed before the data is published (`build.py`), and no engine module reads it. The engine only ever sees metrics, so recommendations are not "told" the answer.

9. Assumptions

- Prices come from `config.py` and are illustrative (for example `s-2-4` ₹1,500/month up to `s-16-64` ₹16,000/month); the same prices apply in all regions.
- Workload shapes are hand-designed, not fitted to a real trace.
- Gaussian noise is a stand-in for real measurement variation.
- Launch dates are simulated.

10. Limitations

1. **Weekday behaviour is thin.** The general formula includes a weekday factor, but only `office_dev` and `dev-search-04` actually change on weekends.
2. **Network and IOPS depend on CPU.** They are computed from CPU activity, so the idle rule "three signals must agree" is satisfied by construction in simulated data. Real data would test it properly.
3. **Orphan grace period is not exercised.** Every volume and IP was detached long before

the 7-day grace period could matter, so no orphan sits inside it.

4. **Hardcoded gap VMs.** The three gap VM ids are fixed; if the `DESIGN` table changes, those ids will point at different personas.
5. **Single global random generator.** See section 2.
6. **Simulation only.** CPU realism is cross-checked against the Azure Public Dataset V2 (58% of real VMs average 10% CPU or less, versus 50% here), but Azure has no memory, disk or network data, so only CPU is benchmarked.

11. How to regenerate

```
pip install -r engine/requirements.txt
npm run data        # runs engine/generate.py then engine/build.py
```